Cifar10 using CNN

In [21]:
```
# Import required libraries
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# Set random seed for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Step 1: Load CIFAR-10 Dataset

In [22]:
```
# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

# Class labels
class_names = ['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer',
               'Dog', 'Frog', 'Horse', 'Ship', 'Truck']

print(f"Training data shape: {x_train.shape}")
print(f"Training labels shape: {y_train.shape}")
print(f"Test data shape: {x_test.shape}")
print(f"Number of classes: {len(class_names)}")
```

```
c:\Users\kk817\anaconda3\Lib\site-
packages\keras\src\datasets\cifar.py:18: VisibleDeprecationWarning:
dtype(): align should be passed as Python or NumPy boolean but got
`align=0`. Did you mean to pass a tuple to create a subarray type?
(Deprecated NumPy 2.4)
  d = cPickle.load(f, encoding="bytes")
```

```
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000, 1)
Test data shape: (10000, 32, 32, 3)
Number of classes: 10
```

Step 2: Visualize Sample Images

In [23]:
```
# Visualize 10 sample images
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Step 3: Normalize the Data

In [24]:
```
# Normalize pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Convert labels to one-hot encoding
y_train_encoded = keras.utils.to_categorical(y_train, 10)
y_test_encoded = keras.utils.to_categorical(y_test, 10)

print(f"Training data normalized: min={x_train.min()}, max={x_train.max()}")
print(f"Label shape after encoding: {y_train_encoded.shape}")
```

```
Training data normalized: min=0.0, max=1.0
Label shape after encoding: (50000, 10)
```

Step 4: Build CNN Model

In [25]:

```
# Build a simple CNN model
model = models.Sequential([
    # First Conv Block
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Second Conv Block
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Third Conv Block
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Fully connected layers
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

# Print model summary
model.summary()
```

```
c:\Users\kk817\anaconda3\Lib\site-
packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do
not pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 30, 30, 32)	896
batch_normalization_12 (BatchNormalization)	(None, 30, 30, 32)	128
conv2d_11 (Conv2D)	(None, 28, 28, 32)	9,248

batch_normalization_13 (BatchNormalization)	(None, 28, 28, 32)	128
max_pooling2d_6 (MaxPooling2D)	(None, 14, 14, 32)	0
dropout_8 (Dropout)	(None, 14, 14, 32)	0
conv2d_12 (Conv2D)	(None, 12, 12, 64)	18,496
batch_normalization_14 (BatchNormalization)	(None, 12, 12, 64)	256
conv2d_13 (Conv2D)	(None, 10, 10, 64)	36,928
batch_normalization_15 (BatchNormalization)	(None, 10, 10, 64)	256
max_pooling2d_7 (MaxPooling2D)	(None, 5, 5, 64)	0
dropout_9 (Dropout)	(None, 5, 5, 64)	0
conv2d_14 (Conv2D)	(None, 3, 3, 128)	73,856
batch_normalization_16 (BatchNormalization)	(None, 3, 3, 128)	512
max_pooling2d_8 (MaxPooling2D)	(None, 1, 1, 128)	0
dropout_10 (Dropout)	(None, 1, 1, 128)	0
flatten_2 (Flatten)	(None, 128)	0
dense_4 (Dense)	(None, 256)	33,024
batch_normalization_17 (BatchNormalization)	(None, 256)	1,024
dropout_11 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 10)	2,570

Total params: 177,322 (692.66 KB)

Trainable params: 176,170 (688.16 KB)

Non-trainable params: 1,152 (4.50 KB)

Step 5: Compile the Model

In [26]:

```
# Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

Step 6: Train the Model

In [27]:

```
# Train the model
early_stopping = keras.callbacks.EarlyStopping(monitor='val_loss', patience=3,
restore_best_weights=True)

history = model.fit(
    x_train, y_train_encoded,
    batch_size=64,
    epochs=30,
    validation_split=0.2,
    verbose=1,
    callbacks=[early_stopping]
)

print("\nTraining completed!")
```

```
Epoch 1/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m32s[0m 43ms/step
- accuracy: 0.3203 - loss: 2.0234 - val_accuracy: 0.4340 - val_loss: 1.5242
Epoch 2/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
- accuracy: 0.4976 - loss: 1.4034 - val_accuracy: 0.5592 - val_loss: 1.2206
Epoch 3/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
- accuracy: 0.5642 - loss: 1.2173 - val_accuracy: 0.5479 - val_loss: 1.2925
Epoch 4/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
- accuracy: 0.6115 - loss: 1.1019 - val_accuracy: 0.6242 - val_loss: 1.0517
Epoch 5/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m25s[0m 40ms/step
- accuracy: 0.6424 - loss: 1.0176 - val_accuracy: 0.6711 - val_loss: 0.9315
Epoch 6/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m25s[0m 41ms/step
- accuracy: 0.6632 - loss: 0.9648 - val_accuracy: 0.6592 - val_loss: 0.9672
Epoch 7/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m25s[0m 40ms/step
- accuracy: 0.6807 - loss: 0.9149 - val_accuracy: 0.6878 - val_loss: 0.8973
Epoch 8/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m25s[0m 40ms/step
- accuracy: 0.6964 - loss: 0.8759 - val_accuracy: 0.7149 - val_loss: 0.8212
Epoch 9/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m41s[0m 41ms/step
- accuracy: 0.7099 - loss: 0.8353 - val_accuracy: 0.7170 - val_loss: 0.8121
Epoch 10/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
- accuracy: 0.7230 - loss: 0.8027 - val_accuracy: 0.7177 - val_loss: 0.8116
Epoch 11/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
- accuracy: 0.7332 - loss: 0.7734 - val_accuracy: 0.6827 - val_loss: 0.9096
Epoch 12/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
- accuracy: 0.7406 - loss: 0.7523 - val_accuracy: 0.7414 - val_loss: 0.7456
Epoch 13/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
```

```
- accuracy: 0.7507 - loss: 0.7257 - val_accuracy: 0.7467 - val_loss: 0.7278
Epoch 14/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
- accuracy: 0.7547 - loss: 0.7125 - val_accuracy: 0.7484 - val_loss: 0.7374
Epoch 15/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 42ms/step
- accuracy: 0.7624 - loss: 0.6916 - val_accuracy: 0.7312 - val_loss: 0.7980
Epoch 16/30
[1m625/625[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m26s[0m 41ms/step
- accuracy: 0.7673 - loss: 0.6750 - val_accuracy: 0.7074 - val_loss: 0.8830

Training completed!
```

Step 7: Evaluate the Model

In [28]:

```
# Evaluate on test set
test_loss, test_accuracy = model.evaluate(x_test, y_test_encoded, verbose=0)
print(f"\nTest Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)")
```

```
Test Loss: 0.7490
Test Accuracy: 0.7445 (74.45%)
```

Step 8: Plot Training History

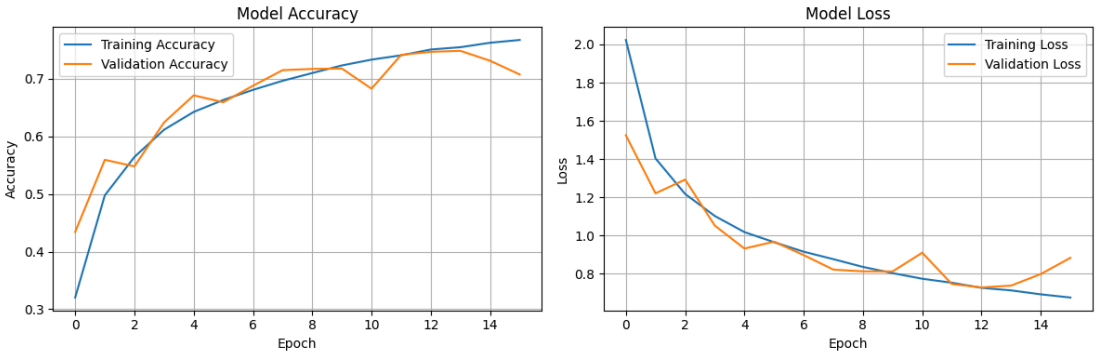
In [29]:

```
# Plot training and validation accuracy
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Accuracy plot
axes[0].plot(history.history['accuracy'], label='Training Accuracy')
axes[0].plot(history.history['val_accuracy'], label='Validation Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Model Accuracy')
axes[0].legend()
axes[0].grid(True)

# Loss plot
axes[1].plot(history.history['loss'], label='Training Loss')
axes[1].plot(history.history['val_loss'], label='Validation Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].set_title('Model Loss')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



Step 9: Make Predictions on Test Images

In [30]:

```
# Make predictions on test images
predictions = model.predict(x_test[:10])

# Display results
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
axes = axes.flatten()

for i in range(10):
    # Get prediction
    predicted_class = np.argmax(predictions[i])
    true_class = y_test[i][0]
    confidence = predictions[i][predicted_class]

    # Plot image
    axes[i].imshow(x_test[i])

    # Set title with predictions
    color = 'green' if predicted_class == true_class else 'red'
    title = f"True: {class_names[true_class]}\nPred:
{class_names[predicted_class]}\n({confidence:.2%})"
    axes[i].set_title(title, color=color, fontsize=9)
    axes[i].axis('off')

plt.tight_layout()
plt.show()
```

```
[1m1/1[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m 185ms/step
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.